Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Field	Details
Date	18 June 2026
Team ID	24EG105H22
Project Name	HouseHunt – House Rent Management System
Maximum Marks	4 Marks

Functional Requirements

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	User Registration	• Register as Owner or Renter via email, name, password • Password hashed with bcryptjs • Role assigned at registration
FR-2	User Authentication	• Login with email and password • JWT token issued on successful login • Token stored in localStorage • Logout clears token
FR-3	Property Listing (Owner)	• Owner can create a new property listing (title, description, address, city, state, price, type, bedrooms, bathrooms, area, amenities, images) • Owner can edit existing listings • Owner can delete listings • New listings start in 'pending' status
FR-4	Property Approval (Admin)	• Admin can view all pending property listings • Admin can approve a property (status → approved, visible publicly) • Admin can reject a property with reason • Only approved properties appear in public search
FR-5	Property Search & Browse (Renter/Public)	• Public can browse all approved properties • Filter by city/location, property type, price range, number of bedrooms • View property detail page with full info
FR-6	Booking Request (Renter)	• Logged-in Renter can submit a booking request for any approved property • Booking captures move-in date and message • Booking starts with 'pending' status

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-7	Booking Management (Owner)	• Owner can view all booking requests for their properties • Owner can approve or reject a booking request • Approved booking changes status to 'approved'
FR-8	Booking Tracking (Renter)	• Renter can view all their submitted booking requests • Status shown: Pending / Approved / Rejected
FR-9	Owner Dashboard	• View all owned properties with their statuses • Quick stats: total properties, pending bookings • Manage incoming booking requests
FR-10	Admin Dashboard	• View platform-wide stats: total users, properties, bookings, pending approvals • List all users • Approve/reject pending properties

Non-Functional Requirements

NFR No.	Non-Functional Requirement	Description & Implementation
NFR-1	Usability	Responsive UI built with Tailwind CSS and shadcn/ui components. Intuitive navigation with role-specific menus. Mobile-friendly layout. Clear error messages and form validation feedback.
NFR-2	Security	JWT-based stateless authentication. Passwords hashed with bcryptjs (salt rounds=10). Role-based access control enforced server-side. Environment secrets managed via Replit secrets. Zod input validation on all API endpoints.
NFR-3	Reliability	Express 5 error handling middleware catches unhandled errors. Database transactions ensure data consistency. pino logger for structured server-side logging. Graceful server shutdown on SIGTERM.
NFR-4	Performance	React Query caches API responses, reducing redundant network calls. Vite build tool with tree-shaking for optimized

NFR No.	Non-Functional Requirement	Description & Implementation
		frontend bundle. esbuild bundles the Express server for fast cold starts. PostgreSQL indexes on frequently queried columns (status, owner_id, city).
NFR-5	Availability	Stateless JWT auth enables horizontal scaling. Hosted on Replit with auto-restart on crashes. Database on managed PostgreSQL (always available). Frontend served via Vite dev server / production build on CDN-ready static hosting.
NFR-6	Scalability	3-tier architecture separates concerns for independent scaling. Stateless API server can be replicated. PostgreSQL supports read replicas. pnpm monorepo structure allows extracting services independently. OpenAPI contract enables new clients (mobile app) without backend changes.
NFR-7	Maintainability	Contract-first OpenAPI spec drives code generation. TypeScript throughout with strict type checking. Drizzle ORM with schema-as-code. Modular route handlers (auth.ts, properties.ts, bookings.ts, admin.ts). Shared lib packages avoid duplication.